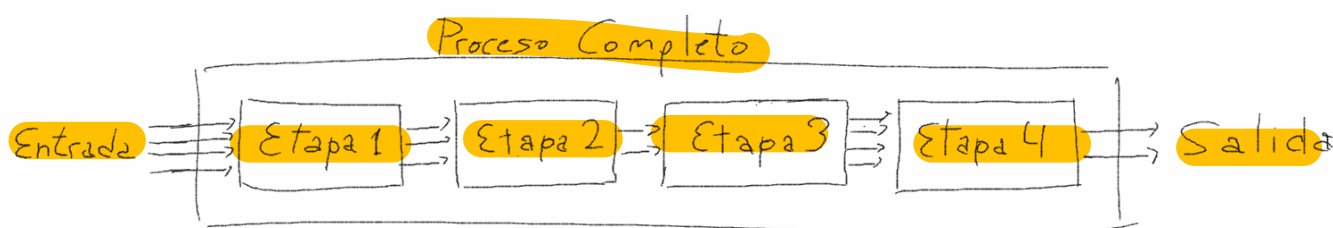


El Pipeline de Renderizado

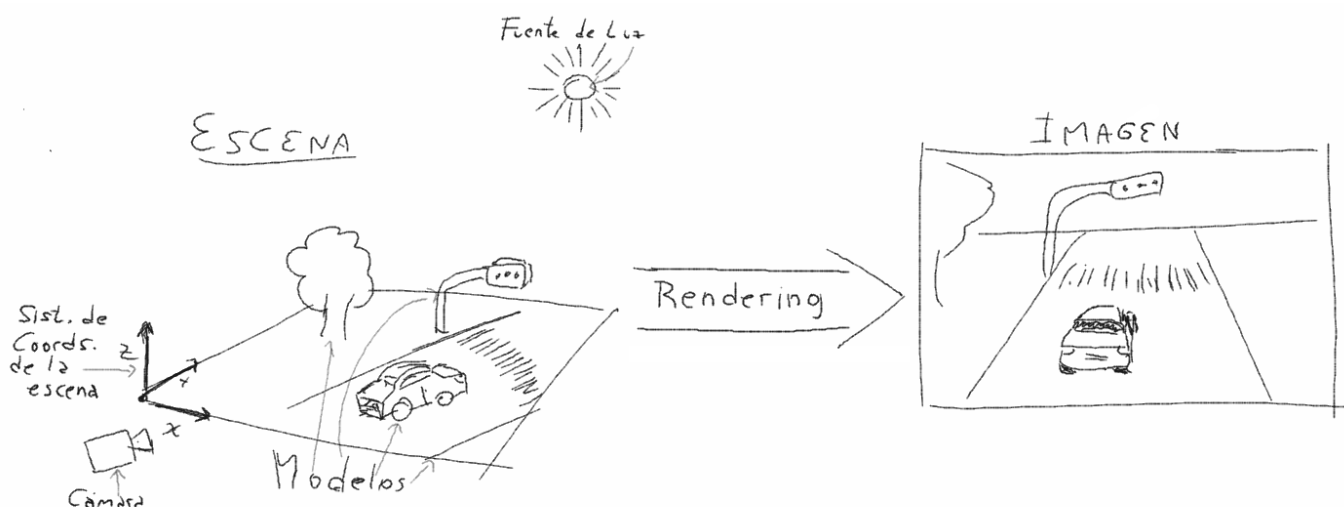
El principal objetivo de este texto es introducir el concepto fundamental de *pipeline de renderizado* que resulta transversal a todo el contenido de la materia. Pero además de dar todas las definiciones fundamentales para comenzar a entender cómo funciona y cómo se programa una aplicación gráfica, este texto referencia a todas las unidades temáticas de la materia a lo largo de su desarrollo, para ayudar a que el alumno vea y estudie cada una como una pieza de un gran rompecabezas y no como temas estancos e independientes. El entender cómo encaja cada tema en el mapa global ayuda mucho a darle sentido a esos contenidos. Se espera que el alumno lea este apunte al comienzo de la materia para empezar a armarse una idea general (llena de agujeros que iremos tapando de a poco) y obtener además las definiciones básicas (vocabulario) que utilizaremos en todas las unidades. Pero se espera también que el alumno vuelva a este mismo apunte al final de la materia para corroborar si las piezas encajaron correctamente. Al final de cursado, casi todo este contenido debería resultar bastante obvio o elemental (aunque por experiencia sabemos que muchas veces no es así).

1. ¿Qué es El Pipeline?

Un **pipeline** ("pipeline"="tubería" en Inglés) es una **secuencia de etapas de algún proceso**, encadenadas de forma que **la salida de una etapa es la entrada de la siguiente**. La entrada de la primera etapa es la entrada del proceso o sistema en general, y análogamente la salida de la última etapa es la salida del mismo.



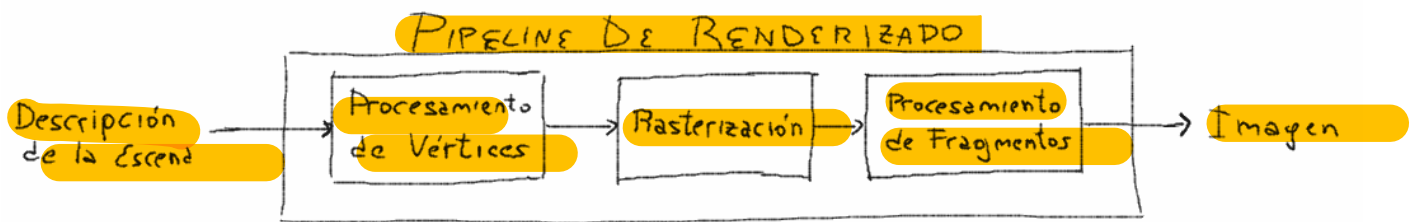
El proceso de **rendering** es el proceso de generar una imagen raster (una matriz de píxeles/puntitos de colores) a partir de la descripción de una escena. La descripción incluye principalmente la **forma de los objetos**, **propiedades de sus materiales** (incluyendo **colores y/o texturas**), **disposición y propiedades de las fuentes de luz**, y **disposición y propiedades de la cámara**.



Se conoce como **pipeline gráfico** o **pipeline de renderizado** (*rendering pipeline*) a un conjunto de etapas más o menos estándar que muchas APIs¹ gráficas utilizan para *renderizar* (generar una imagen).

Hay, simplificando mucho, dos grandes grupos de algoritmos de renderizado: los basados en *rasterización* y los basados en *raytracing*. Ya entraremos en detalle sobre estos conceptos, pero por ahora podemos decir que para gráficos en tiempo real² se utilizan mayoritariamente algoritmos basados en *rasterización* (más adelante veremos qué significa eso), y por eso cuando hablamos del *pipeline de renderizado* pensamos en el conjunto de etapas necesarias para este tipo de algoritmos.

La versión más simplista posible tiene tres etapas principales que nos interesa estudiar en detalle: el procesamiento de vértices, la *rasterización*, y el procesamiento de fragmentos. Pero para entender mejor estas etapas, primero debemos entender cómo se describe la forma de los objetos.



2. Descripción de la geometría: las Primitivas

Para calcular dónde y cómo se verá un objeto en la imagen renderizada, es imprescindible describir la forma/geometría del mismo. Debido a los cálculos que habrá que hacer con el objeto a lo largo del pipeline, resulta conveniente aproximar la superficie real del mismo mediante *primitivas*. Una **primitiva** es una forma geométrica muy simple que se usa para representar de manera aproximada una partecita del objeto. Por ejemplo, podemos aproximar un círculo con un polígono regular de muchos lados (¿un *muchógono*?). En este caso, los lados/segmentos serían las *primitivas*. Si usamos un *cuadrilátero* o un *pentágono*, poco se parecerá el resultado a un círculo; pero si usamos un *isodécágono* (el nombre no importa, pero este tiene 20 lados) se parecerá mucho más. Cuanto mayor el número de lados, mejor aproxima, pero también mayor cantidad de *primitivas* a almacenar y procesar en el *pipeline*.

Cuando se quiere pintar simplemente un punto, la primitiva puede ser el punto mismo. Cuando se describe una curva o el contorno de una superficie, la primitiva es el segmento de recta. Cuando se describe una superficie, las primitivas son mayormente triángulos (y a veces cuadriláteros). En computación gráfica no se usan primitivas de volumen (en mecánica computacional, por ejemplo, podría ser un tetraedro o un hexaedro) porque de los objetos casi siempre vemos solo la superficie (aún si los partimos, solo vemos una nueva superficie interna).

La principal entrada al *pipeline de renderizado* (la que no puede faltar) es una lista de *primitivas*. Supongamos, por ejemplo, que representamos un objeto con 100 triángulos. La entrada puede tomar dos formas: una lista con las coordenadas de 300 puntos (por cada 3 forman un triángulo); o mejor aún una lista de puntos (menos de 300, ya que triángulos "vecinos" comparten puntos) y otra de *conectividades* (por cada triángulo, 3 índices para seleccionar y conectar 3 puntos de la primera lista).

¹Una API es un "interfaz" para programar. Es decir, los prototipos de funciones y las estructuras de datos asociadas para interactuar con una biblioteca. Para controlar directamente una GPU, las API más utilizadas son *OpenGL*, *DirectX* y *Vulkan*. En esta materia utilizaremos *OpenGL moderno*. Lo de "moderno" es porque hay diferencias sustanciales e irreconciliables entre las 2 primeras versiones (que se dicen *OpenGL "viejo"* o *"legacy"*), y las posteriores.

²Se dice "en tiempo real" al renderizado que debe hacerse *rápido*, en un tiempo acotado (por ej, un juego debe poder renderizar al menos 30 veces por segundo, idealmente 60, para dar la sensación de movimiento fluido). En contraposición, el renderizado *off-line* puede usar algoritmos mucho más lentos para, a cambio, generar imágenes mucho más realistas (por ej, para el caso de una película animada, cada fotograma puede tomar días).

En cualquiera de los casos, a los puntos que definen los triángulos los llamaremos **vértices**, y les asociaremos luego más información a cada uno, además de sus coordenadas. Entre la información adicional, una de las cosas más importantes, y que tiene que ver también con la forma del objeto, son las **normales**. Por cada *vértice*, que es un punto de una superficie que se está queriendo aproximar, suele ser útil saber cuál es la normal de la superficie en ese punto (un vector perpendicular a la misma). La utilidad tiene que ver con calcular la iluminación principalmente, entraremos en esto más adelante. Pero lo importante ahora es que lo habitual es describir una geometría mediante un lista de *vértices* (que tienen cada uno mínimamente posición y normal, puede haber más información por *vértice*) y otra de *conectividades* (ternas de índices que dicen cuáles *vértices* conectar para formar cada triángulo). A esta información en conjunto se la denomina **mall**.

3. Vector vs Raster

Los puntos y vectores que ingresan al *pipeline* pertenecen al reino de la información **vectorial**. Es decir, información que está definida en un espacio *continuo*. Dado por ejemplo un sistema de referencia para un espacio 3D (o sea, un punto que oficie de origen y 3 versores para definir los ejes x , y , z), el valor de una componente de un vector, o de una coordenada de un punto puede ser cualquier valor real (de aquí lo de *continuo*).

La imagen que obtenemos de salida, en cambio, pertenece al reino de lo **raster** o *discreto*. Una imagen *raster* no es más que una matriz de píxeles, donde puedo definir el color de cada píxel de forma independiente. La ubicación de un píxel en esta matriz que es la imagen se define mediante un índice de fila y otro de columna, digamos i , j . Es posible hablar de la 3ra fila ($i = 2$ en base 0) de la matriz, o de la 4ta ($i = 3$), pero no de algo intermedio (no vale $i = 3.5$, pues i debe ser entero). Esto es como pensar que partimos nuestra imagen en esa grilla de cuadraditos (el *píxel* es más como una pequeña área que como un punto) y pintamos cada uno con el color de lo que se vea dentro³.

La entrada al *pipeline* es mayormente información *vectorial*, pero la salida es *raster*. En el medio, hay una etapa importantísima que convierte una cosa en otra y es la *rasterización* (tan importante que es el nombre de la etapa, pero a la vez le da el mismo a todo el conjunto de algoritmos de este tipo).

4. Las Etapas Básicas del Pipeline

La visión más básica del *pipeline* tiene **3 etapas principales: (1) procesamiento de vértices, (2) rasterización, y (3) procesamiento de fragmentos**. Todas estas etapas se ejecutan en la GPU (la placa gráfica, un hardware dedicado para esto). La CPU "solamente" se encarga de producir las entradas del proceso (por ejemplo, vértices y conectividades) para enviarlas a la GPU.

En cualquier arquitectura gráfica moderna las etapas 1 y 3 son programables. En esas etapas hay unas pocas operaciones fijas que la GPU realiza siempre sí o sí, pero la mayor parte de estas etapas deben ser implementadas por el programador utilizando algún lenguaje de programación específico para GPUs⁴. Solemos referirnos a estos programas como **shaders**, por ello es común referirse a la etapa 1 como la etapa del *vertex shader* y a la 3 como la de *fragment shader*. La etapa 2 es la única etapa fija que el desarrollador no necesita programar por sí mismo.

Veamos a continuación cuál es la función principal o habitual de cada una, y cómo se relacionan con las unidades temáticas de la materia.

³En los algoritmos más simples generalmente pintaremos cada píxel de acuerdo a lo que "se ve" u ocurre justo en el centro del mismo, y entonces trataremos cada uno casi como un "punto" en la imagen. Pero en realidad el píxel es un cuadrado, tiene un área (una muy pequeña, pero no es un punto), y para ser más correctos/realistas deberíamos promediar todo lo que "se vea" ahí dentro (pero eso es obviamente más costoso).

⁴En esta materia utilizaremos un lenguaje llamado GLSL, que resultará simple de entender porque tiene muchas similitudes con C y C++. Otro lenguaje de *shading* muy utilizado es HLSL (cuando se usa DirectX como API general).

4.1. El procesamiento de vértices

La principal función de esta etapa es "transformar" los vértices para que luego sirvan de entrada para la etapa de rasterización. Supongamos que "dibujamos" un objeto en algún software de diseño 3D y lo queremos usar en nuestra escena. El modelo tendrá sus vértices definidos en algún sistema de coordenadas arbitrario que haya resultado cómodo para su diseño (usualmente estará centrado o apoyado sobre el origen) y tendrá también una escala y orientación arbitrarias. Cuando hay que componer una escena sumando varios modelos, se define un sistema de coordenadas "global" para toda la escena, y en ese sistema se debe ubicar a cada modelo. Es decir, a cada modelo se lo escala, rota y desplaza (estas son las 3 **transformaciones** más habituales) para acomodarlo en relación a los demás.

Transformar un modelo que está representado por una *mall*a no es más que transformar cada uno de sus vértices. Y esta es la principal función de esta etapa. El programa al que denominamos *vertex shader* se ejecuta una vez por cada vértice que se envíe a la GPU, y en su ejecución lo modifica (modifica por ej las coordenadas de su posición y las componentes de su normal asociada). No diremos todavía por qué, pero es conveniente por razones de eficiencia representar a las transformaciones mediante matrices de 4×4 ⁵.

Es responsabilidad del programador escribir un *vertex shader* que transforme el objeto para colocarlo en el mundo (en relación a los demás modelos). Pero no es la única transformación que se aplica, porque el resultado final de esta etapa debe darnos la posición en la que el vértice se verá en la pantalla/ventana de la aplicación. Entonces también se debe tener en cuenta la posición, orientación y otras propiedades de la cámara. Esto se implementa también en el *vertex shader* mediante dos transformaciones/matrices adicionales⁶.

En la unidad de "Espacios y Transformaciones" veremos todos los detalles y fundamentos matemáticos del uso de matrices en general, del significado de la 4ta dimensión, y de las 3 transformaciones principales en particular.

Todo lo descrito hasta ahora debe ser programado por el usuario. Técnicamente en esta etapa además se adapta el resultado final de este programa al tamaño de la ventana, ya que lo programado en realidad deja los vértices en un espacio normalizado (va de -1 a +1 en cada dimensión) que luego hay que "estirar" a lo que ocupe en pantalla. También al final de esta etapa se se descarta todo lo que finalmente quede fuera de la pantalla y entonces no deba ser rasterizado⁷. y se convierte esos datos de 4D a 3D⁸.

Tenemos hasta aquí la funcionalidad básica que encontrará en el 99% de los *vertex shaders*. El programador puede agregar otros pasos y cálculos para hacer trucos varios, el vértice puede tener más información que solo posición y normal que deba transformarse o al menos transferirse a la rasterización, y eventualmente hay otra información general (que no depende de cada vértice) que también puede requerir algún tratamiento⁹.

La salida final de esta etapa son *vértices transformados*, con todo lo que esto implique (cada vértice podrá tener información adicional, como por ej su normal), que conectados definen las *primitivas* (los triángulos).

⁵A esta matriz se la denomina habitualmente *model* porque pasa del sistema de referencia local del modelo al global de la escena (el llamado espacio del mundo o *world space*).

⁶A estas matrices se las suele denominar *view* (la que define dónde va la cámara y hacia dónde mira) y *projection* (codifica otras propiedades de la cámara, como la relación de aspecto o la apertura/perspectiva). *view* transforma los vértices desde el espacio del mundo (*world space*) al del ojo (*view space*), y *projection* mapea todo lo que se debe ver a un cubo que va desde $[-1, -1, -1]$ a $[+1, +1, +1]$ (*NDC* o sistema de coordenadas normalizado).

⁷Esto es lo que se conoce como *culling* (descartar las primitivas que no se ven), y *clipping* (si alguna primitiva se ve a medias, recortarla y quedarse solo con la porción visible). El criterio obvio para decir que una primitiva no se ve y debe descartarse es si queda fuera del volumen visual (del cubo de -1 a +1), pero también se usa otro criterio interesante que se conoce como *cull face* y consiste en descartar primitivas de acuerdo a si la cámara las ve *de frente* o *de espaldas*, y tiene sentido solo para objetos cerrados.

⁸Esta etapa suele aparecer en los diagramas detallados como *perspective division* (división perspectiva). Veremos más adelante por qué, pero para aplicar eficientemente las transformaciones a los vértices, conviene llevarlos a un espacio (llamado *proyectivo*) donde hay una coordenada adicional y bastante especial denominada *w*; por eso al final debemos volver a reducir la dimensionalidad, para volver al espacio 3D "normal".

⁹Por ejemplo, la posición de la luz es la misma para todos los vértices, pero también debe ajustarse de acuerdo a la cámara. Entonces a esta posición también se le pueden aplicar algunas de las transformaciones en esta etapa.

4.2. La rasterización

En esta etapa se pasa del mundo *vectorial* al mundo *raster*. Por cada *primitiva*, la etapa de **rasterización** debe determinar qué píxeles de la imagen cubre o toca, y además *interpolar* las propiedades de los mismos. La entrada entonces son las primitivas (que se construyen conectando los vértices transformados de la etapa anterior), y la salida son *fragmentos*.

Cada **fragmento** se corresponde con un píxel de la imagen que es cubierto por una *primitiva*, pero no es exactamente lo mismo. Decimos que un *fragmento* es un *candidato* a píxel. Esto es así porque el fragmento luego podría descartarse por diferentes motivos (por ej, porque en la imagen final es tapado por otro fragmento de otra *primitiva* que se encuentre más cerca de la cámara). Además, el *fragmento* tiene información adicional. La información mínima es su posición en la matriz de la imagen, pero suele tener asociado uno o varios colores (para definir de qué color pintarlo si finalmente se convierte en píxel), u otras propiedades que permitan luego calcular su color (como por ejemplo la normal si vamos a calcular cómo le rebota la luz), e información adicional varia (la distancia a la cámara, por ejemplo, servirá para saber entre dos *fragmentos* que quieran pintar el mismo píxel cuál tapa a cuál).

Toda la información adicional del *fragmento* se obtiene a partir de la información de los *vértices*. Si por ejemplo los *vértices* tenían normales asociadas, los *fragmentos* se generarán con normales asociadas. Las normales de los *fragmentos* se obtienen por **interpolación**. Esto es, haciendo una mezcla (promedio ponderado) de las normales de los *vértices* a partir de los cuales se generó el fragmento (los de su triángulo, con un criterio que obviamente le de mayor peso en el promedio a el/los *vértices* que estén más cerca del *fragmento*).

En la unidad "Rasterización" veremos los detalles de los algoritmos que sirven para saber qué píxeles/*fragmentos* cubre una *primitiva*. En la unidad "Interpolación" estudiaremos los detalles de la forma de obtener la información adicional de cada *fragmento* (la forma de calcular los promedios ponderados correctamente).

4.3. El procesamiento de fragmentos

En esta etapa se reciben los *fragmentos* que generó la rasterización y se debe determinar cuáles serán finalmente visibles y cuáles serán descartados; y en el caso de los primeros, el color con el que se debe pintar el píxel en la imagen final. En esta etapa hay dos grandes procesos que se realizan a cada uno de los *fragmentos*. El primero determina el color con el que se debería pintar el mismo, lo cual involucra usualmente calcular la interacción con la luz¹⁰ y eventualmente aplicar una *textura* (pegarle una imagen si la primitiva no es toda de un solo color). El segundo determina si efectivamente el *fragmento* será visible o debe descartarse.

Recordemos que un *fragmento* es como un pedacito de una *primitiva* (de un triángulo por ejemplo), que a su vez era un pedacito del modelo (de una superficie). Entonces, el fragmento representa generalmente un pedacito de la superficie del modelo. El modelo será de cierto color en ese lugar. Si el color es fijo, ese color es simplemente un dato general (el mismo para todos) para el *fragment shader*. Si el color del objeto varía sobre la superficie, se suele *pegar* una imagen (**textura**) sobre la misma y obtener el color de ese pedacito a partir de un pedacito de la imagen.

En cualquier caso, el color que se tiene no es necesariamente el que se vería en la imagen final. Si el *fragmento* está, por ejemplo, en un lugar que no recibe luz, seguramente se verá negro o casi negro. Si el *fragmento* corresponde a un objeto metálico, independientemente del color del metal, podría verse casi blanco (donde el metal "brilla"). Para determinar el verdadero color, hay que modelar la interacción de la luz con el *fragmento* (qué luz llega a ese pedacito de superficie, y cómo y cuánto rebota en dirección a la cámara, lo cual depende del tipo de material del objeto).

¹⁰Esto es lo que da origen a la palabra *shading* (sombreado), ya que fue la primera etapa que se tuvo interés en poder programar. Luego el concepto se extendió para referirse a cualquier programa que corre en la GPU, aunque su función no sea *sombrear*, o corresponda a otra etapa del *pipeline*.

Finalmente, resta determinar si el *fragmento* es efectivamente visible, o debe ser descartado. Para esta resolver esta operación por *fragmento* hay un algoritmo muy eficiente conocido *z-buffer* o *depth-buffer*, sin el cual el renderizado basado en rasterización no sería tan eficiente. Este algoritmo utiliza un *buffer* auxiliar (en el que guarda valores de *z* o profundidad para luego compararlos, de ahí su nombre).

Si bien **buffer** en general puede referir a una región de memoria con distintos tipos o estructuras de datos, en el contexto de *rendering*, y más aún en operaciones por *fragmento*, un *buffer* es una matriz que guarda un dato por posición en la imagen final (por pixel). La imagen que vemos, por ejemplo, guarda un color por cada pixel/posición. Se puede pensar en general a los *buffers* como imágenes auxiliares, pero el dato que guardan podría representar cualquier cosa, no solo un color. Utilizando *buffers* auxiliares se pueden hacer ciertos trucos muy útiles y comunes, y resolver ciertos problemas. Por esto algunos de ellos (y sus operaciones) están implementados directamente en el hardware gráfico y solo debemos *configurar* su funcionamiento (en lugar de *programarlo* desde cero en el *fragment-shader*).

Entonces, en esta etapa se "programa" la primera parte en el *fragment-shader* (que consiste principalmente en determinar el color final del *fragmento*, mediante el uso de texturas y el modelado de la interacción con la luz), y solo se "configura" luego el funcionamiento de los *buffers* y sus operaciones asociadas (que llamaremos **tests**, y sirven principalmente para determinar si el fragmento será o no visible en la imagen final)¹¹.

En la unidad "Texturas" veremos los detalles del uso de imágenes para darle color u otras propiedades a los fragmentos (no es trivial por ejemplo decidir cómo "pegar" la imagen sobre el objeto cuando no es plano). En la unidad de "Color e Iluminación" veremos un modelo muy sencillo¹² para calcular la interacción del objeto con la luz. Y finalmente, en la unidad "Técnicas del espacio de la imagen" (a la que también nos referimos como "Buffers"), veremos los detalles del uso de *buffers* y algoritmos auxiliares para el descarte de *fragmentos*, y otros trucos asociados.

4.4. Otras etapas

Hemos descrito las 3 etapas más importantes, tanto desde lo teórico, como desde la programación. Existen otras que en general ignoraremos o bien por ser triviales y no requerir mayor trabajo, o bien por ser complejas y no utilizarse con tanta frecuencia.

En los diagramas más detallados del un *pipeline*, aparece entre el procesamiento de vértices y la rasterización, una etapa de "ensamblado de primitivas", que no es más que el hecho de conectar de a 3 los vértices procesados para formar los triángulos (o de a 2 si por ejemplo se quiere rasterizar segmentos).

Vamos a suponer además la mayoría de las veces que la salida del *fragment shader* conforma la imagen final. Es decir, los pixeles se van pintando a partir de los fragmentos que sobreviven a los *tests*. Técnicamente hay una etapa más denominada **blending**, en la que el color de un *fragmento* se mezcla con lo que tenga previamente la imagen en ese píxel. Esto es así porque, por ejemplo, si el fragmento representa algo semitransparente, no debe simplemente reemplazar el color, sino mezclarse con lo que había "detrás" para dar ese efecto de transparencia.

Por último, hay otras etapas programables que no son imprescindibles, que se las puede omitir por completo y dejar fuera del *pipeline* básico. Existe por ejemplo la posibilidad de programar un *geometry shader* o un *tessellation shader*, que son programas que van antes de la *rasterización* y que toman como entrada *primitivas* (triángulos, no *vértices* sueltos como el *vertex shader*), y pueden modificarlas, subdividir las o generar nuevas *primitivas*. En el desarrollo de

¹¹Puede ser importante mencionar que en un renderizado realista, la etapa de aplicación de textura y cálculo de iluminación suele ser la más compleja y costosa de todo el proceso. Esto haría pensar que conviene primero usar los *buffers/tests* para descartar y luego solo calcular iluminación a lo que sobreviva. Pero dado que el *fragment shader* podría modificar el fragmento y así cambiar el resultado de un *test*, "lógicamente" se ejecuta primero.

¹²Solo estudiaremos en detalle el modelo de iluminación de *Phong*, o su variante *Blinn-Phong*. Comentaremos las bases de modelos más realistas (PBR), pero no tendremos tiempo de profundizar en ellos.

la materia no necesitaremos de estas etapas, y por lo tanto las dejaremos efectivamente fuera de nuestro análisis del *pipeline*.

5. La aplicación

Hasta ahora nos hemos centrado en describir lo que ocurre en la GPU. Pero es la aplicación que corre en CPU la encargada de configurar y alimentar de datos a la GPU. En algunas unidades estudiaremos técnicas que serán necesarias en la aplicación, previo al ingreso al *pipeline*.

Por ejemplo: sería muy poco práctico tener que definir una por una las *primitivas* de una superficie. Hay técnicas para modelar matemáticamente curvas y superficies, y luego formas de, a partir de estos modelos, generar las *primitivas* para alimentar a la GPU. En la unidad de "Curvas y Superficies" estudiaremos algunos de estos modelados.

Además, dado que el tipo de aplicaciones que se asocian a la geometría computacional y a la computación gráfica suelen ser aplicaciones que manejan grandes volúmenes de información (por ejemplo, escenas definidas por miles o millones de *primitivas*), en la unidad de "Intersecciones y Ordenamiento Espacial" introduciremos (aunque sin mucha profundidad) algunas estructuras de datos que se suelen utilizar para acelerar/optimizar los cálculos (en general, para evitar tener que hacer un todos contra todos, por ejemplo, cuando hay que determinar si dos mallas se tocan o intersectan).

Con todo esto, el alumno debería tener una noción básica de cómo funciona una aplicación gráfica interactiva (pues solo desarrollaremos ejemplos muy acotados debido al tiempo limitado del cursado) casi que de principio a fin, y eso sentar las bases para luego poder investigar o desarrollar técnicas o aplicaciones más avanzadas y parecidas a las del "mundo real".